

План разработки Telegram Mini App (CRM для преподавателей)

Спринт 1 — Архитектура и запуск проекта

Цель спринта: получить рабочий каркас — бэкенд подключён к базе, фронтенд стартует как Telegram WebApp и умеет авторизоваться (пока заглушка). Это позволит понять процесс и оценить усилия.

Ожидаемый результат

- Репозиторий с двумя папками: `server/` (NestJS) и `web/` (React + Vite).
- Локальная или удалённая PostgreSQL база, доступная проекту.
- Prisma настроен и выполнена первая миграция с моделями `Teacher` и `Student`.
- Минимальный фронтенд, который при запуске внутри Telegram WebApp показывает "Hello" и отправляет `initData` на бэкенд.

Предварительные требования (коротко и понятно)

1. **Установи Node.js 18+** (скачай с официального сайта).
2. **Git** (для управления кодом).
3. **Docker** (рекомендуется для PostgreSQL, но можно использовать Supabase).
4. Аккаунт **Supabase** (опционально) если не хочешь настраивать Postgres локально.
5. **Telegram Bot** — создай бота через @BotFather и получи токен (понадобится позже для WebApp).

Структура репозитория (как будет выглядеть)

```
mini-crm/  
├─ server/      # NestJS бэкенд  
├─ web/         # React (Vite) фронтенд  
├─ .gitignore  
└─ README.md
```

Шаги — подробная пошаговая инструкция

1) Создать репозиторий и базовую организацию

- В терминале:

```
bash  
mkdir mini-crm && cd mini-crm  
git init  
echo "node_modules/  
.env
```

```
/dist  
" > .gitignore  
git checkout -b dev
```

- Создай в корне `README.md` и опиши коротко цель проекта.

2) Поднять PostgreSQL

Вариант А — локально через Docker (рекомендуется для старта):

```
docker run --name mini-crm-postgres -e POSTGRES_PASSWORD=pass -e  
POSTGRES_USER=app -e POSTGRES_DB=mini_crm -p 5432:5432 -d postgres:15
```

- После запуска: хост `localhost`, порт `5432`, пользователь `app`, пароль `pass`, база `mini_crm`.

Вариант В — Supabase (не нужен Docker): - Зарегистрируйся на supabase.com → создай проект → получишь URL и credentials. Запиши их.

3) Создать бэкенд (NestJS) — простыми командами

- В корне:

```
npx @nestjs/cli new server  
# выбрать npm, базовые настройки  
cd server
```

- Установи зависимости:

```
npm install prisma @prisma/client dotenv  
npm install -D prisma  
npx prisma init
```

- В `server/.env` добавь строку подключения к базе (пример для Docker):

```
DATABASE_URL="postgresql://app:pass@localhost:5432/mini_crm?  
schema=public"
```

4) Описать модели Prisma (файл `prisma/schema.prisma`)

Вместо сложной схемы — сначала две простые модели:

```
generator client {  
  provider = "prisma-client-js"  
}
```

```

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model Teacher {
  id          Int      @id @default(autoincrement())
  telegramId  String   @unique
  name        String?
  createdAt   DateTime @default(now())
  students    Student[]
}

model Student {
  id          Int      @id @default(autoincrement())
  teacherId   Int
  name        String
  balance      Float   @default(0)
  pricePerLesson Float @default(0)
  createdAt   DateTime @default(now())

  teacher Teacher @relation(fields: [teacherId], references: [id])
}

```

- Затем запусти миграцию:

```
npx prisma migrate dev --name init
```

- И сгенерируй клиент Prisma автоматически (команда выполнится вместе с миграцией).

5) Создать минимальный REST API для проверки

- В NestJS создай модуль `teachers` и `students` (CLI или вручную):

```

npx nest g module teachers
npx nest g controller teachers
npx nest g service teachers
npx nest g module students
npx nest g controller students
npx nest g service students

```

- В сервисах используй `PrismaClient` (`@prisma/client`) для простых функций: найти или создать `Teacher` по `telegramId`, CRUD для `Student`.

Пояснение (не пугайся): это просто файлы с функциями типа `createStudent`, `getStudentsForTeacher` которые вызывают Prisma.

- Запусти сервер:

```
npm run start:dev
```

- Убедись, что сервер поднялся на `http://localhost:3000`.

6) Создать фронтенд (React + Vite)

- В корне:

```
npm create vite@latest web -- --template react
cd web
npm install
```

- Для простоты создадим страницу, которая будет показывать "Hello from WebApp".
- В `index.html` или `src/main.jsx` подключение Telegram WebApp обычно происходит автоматически внутри Telegram — для локального теста можно сделать простой заглушечный объект:

```
// src/telegramStub.js (для локальной разработки)
export const Telegram = window.Telegram || { WebApp: { init: () => {},
onEvent: () => {}, initData: null } };
```

- На странице `App.jsx` просто показываем "Hello" и кнопку "Send initData to server" — по нажатию отправляем тестовый `initData` на `POST /auth/telegram`.
- Запусти фронтенд:

```
npm run dev
```

7) Настроить Endpoint `POST /auth/telegram` на бэке (минимальная версия)

- Пока без проверки подписи (на старте можно заглушить проверку) — endpoint должен принимать объект `{ initData }`, извлекать `telegramId`, и создавать `Teacher` в БД если не найден.
- Возвращать простой JSON с `ok: true` и `teacherId`.

Важное замечание: позже мы добавим проверку подписи Telegram (`hash`) — для прототипа можно временно опустить, чтобы быстрее двигаться.

8) Проверка интеграции

- Открой фронт (`http://localhost:5173`) — нажми кнопку отправки `initData`.
- Проверь в логах сервера, что появился запрос к `/auth/telegram` и в БД создался `Teacher`.

9) Git — фиксация прогресса

```
git add .
git commit -m "sprint1: initial scaffold backend + frontend + prisma models"
git push origin dev
```

Советы и распространённые проблемы (и как их быстро решать)

- **Проблемы с подключением к Postgres:** проверь правильно ли указан `DATABASE_URL` в `.env` и запущен ли контейнер Docker.
- **Prisma не видит базу:** удали миграции и запусти `npx prisma migrate reset` (это удалит данные — безопасно для прототипа).
- **CORS:** при запросах с фронта добавь `app.enableCors()` в `main.ts` NestJS.
- **Telegram WebApp в локале:** браузер не создаёт `window.Telegram` — используй заглушку или тестируй WebApp через инструменты Telegram (потом объясню, как запустить WebApp ссылкой).

Что я могу сделать прямо сейчас для тебя

- Подготовить точные команды и шаблон файлов для `server/` (контроллеры, сервисы, пример использования Prisma).
- Подготовить минимальный `App.jsx` для фронта с кнопкой отправки `initData`.
- Помочь настроить Supabase, если не хочешь Docker.

Если хочешь — скажи, что из этого сделать первым, и я сразу добавлю шаблонные файлы в документ проекта.

Спринт 2 — Авторизация через Telegram

— Авторизация через Telegram - Реализовать получение `initData` на фронте - Сделать endpoint `POST /auth/telegram` - Проверить подпись данных - Создать преподавателя при первом входе - Возвращать собственный JWT

Спринт 3 — CRUD для учеников

- Реализовать endpoints:
 - `POST /students`
 - `GET /students`
 - `PATCH /students/:id`
- На фронте реализовать:
 - список учеников
 - форму добавления ученика
 - карточку ученика

Спринт 4 — Баланс и проведение занятий

- Добавить поля: balance, price_per_lesson
- Создать endpoint POST /lessons/complete
- Реализовать списание с баланса
- На фронте:
 - кнопка "Провести занятие"
 - отображение баланса

Спринт 5 — Хостинг и CI/CD

- Развернуть бэкенд на Render/Railway
- Развернуть фронтенд на Vercel
- Подключить базу (Supabase/Neon)
- Встроить WebApp в Telegram
- Настроить Git: ветки, PR

Спринт 6 — Расписание (опционально)

- Добавить сущность расписания
- Интерфейс календаря
- Отображение занятий

Спринт 7 — Статистика и отчёты (опционально)

- История списаний
- Отчёты по ученикам
- Аналитика занятий